

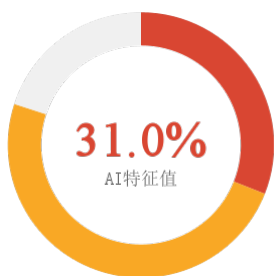
AIGC 检测 · 全文报告单

NO: db9a175246cf47f8b7ca7c9b6e744f67

检测时间: 2026-05-11 17:04:05

篇名: 软2李一凡论文正文基于YOLO与ByteTrack的多路视频目标追踪系统docx_项目化修订版 - 副本.docx

全文检测概览



AI特征值: 31.0%

AI特征显著字符数: 9428

总字符数: 30306

- AI特征显著 (计入AI特征字符数)
- AI特征疑似 (未计入AI特征字符数)
- 未标识部分

分段检测结果

序号	AI特征值	AI特征字符数/章节(部分)字符数	章节(部分)名称
1	31.11%	9428 / 30306	内容1
2	0.00%	0 / 0	参考文献

分段检测详情

1. 内容1

AI特征值: 31.11% AI特征字符数/章节字符数: 9428 / 30306

题目

0.01% (2)

Subject

Multi-video Object Tracking System Based on YOLO and ByteTrack

0.20% (62)

学 院:

0.11% (33)

专业班级:

姓 名:

0.11% (33)

学 号:

0.11% (33)

指导教师:

0.10% (31)

完成日期:

0.10% (31)

教务处制

摘 要

0.02% (6)

本课题围绕多路视频目标追踪的工程实现展开。项目中需要同时处理本地视频、摄像头或网络流，并在同一套程序内完成 YOLO 检测、ByteTrack 身份关联、轨迹绘制、ROI 区域计数以及性能记录。在实际调试中，系统容易遇到几个很具体的问题：多路输入速度不一致时批次形成会等待，推理结果分发如果缺少帧元信息容易错配视频源，ByteTrack 对帧序比较敏感，显示与保存模块也会放大前面环节的延迟。针对这些问题，本文在 Tracking-by-Detection 流程上设计并实现了多视频批处理目标追踪系统。系统通过 FrameData 记录 video_id、video_index、frame_id、timestamp 和检测结果，用 BatchCollector 按 k 值从各视频队列收集帧，再由 ResultDistributor 依据 FrameMeta 将推理结果写回对应视频流，保证检测、追踪和可视化之间的数据对应关系。对于共享队列产生的乱序场景，系统采用以帧序号为键的最小堆缓冲，使同一路视频在进入 ByteTrack 前恢复原始时间顺序。实验部分比较了批内分发与最小堆重排两类策略，并测试 TensorRT engine、输入分辨率和显存占用等因素。结果表明，TensorRT engine 能明显降低检测推理时间，较小单批次容量更适合在线低延迟处理，1280 分辨率对远景小目标更友好，而集中式批推理能够减少重复加载模型带来的资源浪费。该系统为多路视频实时分析提供了一个可运行、可观测、便于继续扩展的实现样例。

关键词：YOLO；ByteTrack；多目标追踪；多路视频；乱序恢复；优先队列；批处理推理；TensorRT

目 录

0.02% (6)

第 1 章 绪论

0.03% (8)

1.1 研究背景

0.03% (8)

近几年，校园、道路、商场和园区中的摄像头数量持续增加，视频已经不只是事后回看的资料，也逐渐成为实时管理和数据分析的入口。和文字记录或单个传感器相比，视频能保留目标外观、位置、运动方向和场景上下文，这些信息对异常发现、流量统计和轨迹分析都很有价值。仅知道画面里有没有人或车还不够，很多场景更关心目标在连续帧中是否还是同一个、经过了哪些区域、停留了多长时间。因此，把目标检测和多目标追踪结合起来，形成可实时运行的视频分析系统，是本课题选择 YOLO 与ByteTrack 作为基础方案的主要原因 [1]，[2]。

0.84% (255)

从应用上看，目标追踪并不是单纯给画面画框。安防场景需要根据轨迹判断聚集、闯入和滞留；交通场景需要连续统计车辆或行人的通行情况；校园或展馆这类区域则更关注不同入口、通道和重点区域的人流变化。人工观察可以处理少量视频，但当视频源变多、运行时间变长以后，人工方式很难持续输出稳定的编号、轨迹和区域统计结果。本文系统中的 ROI 计数、轨迹线绘制和多视频显示，正是围绕这些实际需求设计的。

0.63% (190)

传统监控依赖人工值守和回看，在小规模场景下还能使用，但放到多摄像头并发任务里问题会变得明显。值守人员长时间盯屏容易疲劳，关键帧可能被漏掉；回看又具有滞后性，无法满足实时处置。更重要的是，人工判断很难把每个目标的坐标、编号和轨迹连续记录下来，也就难以支撑后续检索、统计和对比分析。

0.46% (139)

此外，传统人工监控难以稳定输出结构化和可量化的分析结果。具体到本项目，例如，在区域人数统计、目标轨迹分析和客流变化评估等任务中，人工观察往往只能给出经验性判断，难以持续记录每个目标的编号、位置变化和运动轨迹。对于需要长期运行的监控系统而言，不同人员、不同时段之间的判断标准也难以保持一致。相比之下，基于深度学习的目标检测与多目标追踪技术能够将非结构化视频转化为目标类别、位置坐标、置信度、track_id和运动轨迹等结构化信息，从而明显提高视频分析的自动化水平和数据利用效率。 [1]

0.80% (242)

YOLO 系列模型在速度和精度之间取得了较好的平衡，适合作为视频帧检测器 [3]；ByteTrack 通过同时利用高、低置信度检测框，能在一定程度上缓解遮挡或短时漏检造成的轨迹断裂 [5]。不过，在本项目的多路视频环境里，真正麻烦的并不只是算法调用。多个读取线程、批处理收集、结果分发和追踪器状态之间存在严格的数据对应关系，一旦帧序或 video_index 混乱，检测结果就可能被送到错误的视频流，ByteTrack 的 ID 也会随之不稳定。

因此，面向实际应用场景的实时视频多目标追踪系统，不仅需要选择合适的检测器和追踪器，还需要从系统架构层面对多路视频调度、帧序一致性、批处理推理、结果可视化和性能监控进行综合设计。本文正是在这一背景下，围绕多路视频实时处理中的检测效率、追踪稳定性和系统可扩展性问题，研究并实现基于 YOLO 与 ByteTrack 的多路视频批处理目标追踪系统，为智能视频分析系统的工程化落地提供一种可行方案。 [1]，[3]，[5]

0.69% (209)

1.2 研究目的与意义

0.04% (11)

本文的目标是完成一套能在本地运行的多路视频目标追踪系统。系统需要支持多视频输入，完成 YOLO 检测、ByteTrack 追踪、轨迹与检测框显示、ROI 区域统计、结果视频保存和性能监控。实现时重点关注三个工程问题：第一，如何让多个视频源共享批量推理能力；第二，如何在分发检测结果时保持视频源和帧序对应；第三，如何把延迟、FPS、显存等运行状态记录下来，便于后续调参和实验分析 [1], [3], [5]。

0.67% (203)

具体而言，本研究希望解决三个方面的问题。具体到本项目，第一，在算法应用层面，实现检测模块和追踪模块的有效协同，使系统能够在复杂背景、目标遮挡和多目标并存的场景中保持较稳定的检测与追踪效果 [1]。第二，在系统架构层面，设计适用于多路视频并发输入的处理流程，通过生产者-消费者模型将视频读取、批量推理、目标追踪和结果输出等环节解耦，减少不同处理阶段之间的相互阻塞 [8]。第三，在工程优化层面，针对多线程环境下可能出现的帧序错乱问题设计乱序恢复机制，并通过多视频批处理推理流水线提升 GPU 计算资源利用率 [9]。

0.85% (257)

从方法设计角度看，本文围绕 Tracking-by-Detection 范式下检测器、追踪器与并发系统之间的协同关系展开研究 [1], [5]。具体到本项目，传统的多目标追踪研究往往更加关注检测精度或数据关联算法本身，而在实际系统中，视频帧的输入顺序、推理调度方式和线程间数据传递机制同样会影响追踪结果的稳定性 [5], [8]。本文将帧序一致性问题纳入多目标追踪系统设计过程，提出基于优先队列的乱序帧恢复方案，并继续设计面向多视频的批处理推理机制，为实时视觉系统中的时序管理与资源调度提供了一个工程化参考。

0.84% (254)

从落地使用角度看，本课题实现的系统可用于智能安防、交通管理、校园安全和商业客流统计等场景。系统能够自动输出目标检测框、track_id、运动轨迹和区域计数结果，减少人工监控压力，提高视频分析效率。同时，系统中的性能监控模块能够对帧处理延迟和平均处理速度进行实时观测，为后续参数调优和硬件部署提供数据依据。通过对检测、追踪、显示和监控功能的集成，本文研究有助于提升视频监控系统的智能化、自动化和可维护性。 [1], [2]

0.69% (210)

1.3 本章小结

0.03% (8)

本章说明了课题来源、应用需求和主要问题。后续章节将围绕 YOLO 检测、ByteTrack 追踪、多视频批处理、最小堆乱序恢复和性能实验展开，重点不是单独讨论某一个算法，而是说明这些算法和工程模块如何在同一套系统中配合运行。

0.37% (112)

第 2 章 相关技术基础

0.04% (12)

2.1 多目标追踪任务概述

0.04% (13)

多目标追踪 (Multiple Object Tracking, MOT) 是计算机视觉领域中面向视频序列的重要任务，其基本目标是在连续视频帧中同时定位多个感兴趣目标，并在时间维度上为同一物理目标维持稳定的身份标识 [1], [2]。与单帧目标检测只关注当前帧中有哪些目标以及目标位于何处不同，多目标追踪还需要判断不同帧中的哪些检测输出属于同一个目标。因此，MOT 不仅涉及目标空间位置估计，还涉及跨帧数据关联和目标身份管理 [1]。

0.71% (214)

在典型视频处理系统中，多目标追踪的输入通常为按时间顺序排列的视频帧，或由目标检测器输出的逐帧检测输出。检测输出一般包括检测框、目标类别、置信度等信息。其中，检测框用于描述目标在图像坐标系中的位置，目标类别表示检测器判断该目标所属的语义类型，如 person、car、bus 等；置信度表示模型对该检测输出可靠性的估计。追踪算法在这些检测输出的基础上继续分配 track_id，即轨迹编号，用于标识跨帧连续出现的同一目标。 [1]

0.71% (214)

多目标追踪的输出通常是一组带有时间属性的轨迹。每条轨迹由若干帧中的目标位置组成，并关联同一个 track_id。轨迹不仅能够用于可视化目标运动路径，还能够支撑区域计数、行为分析、目标检索、异常检测等上层任务。例如，在视频目标搜索场景中，系统需要根据目标类别、出现位置或运动轨迹对视频内容进行检索和分析；在交通监控场景中，轨迹可用于车辆流量统计、拥堵分析和违规行为识别。 [1], [2]

0.64% (193)

评价多目标追踪效果时，需要关注检测质量和身份保持能力两个方面。具体到本项目，漏检是指真实存在的目标没有被检测或追踪到，会导致轨迹中断或目标消失；误检是指算法将背景或无关区域错误识别为目标，会引入虚假轨迹；ID switch 是指同一物理目标在追踪过程中被错误分配为另一个 track_id，或两个目标之间发生身份交换。对于工程系统而言，ID switch 会直接影响轨迹连续性和目标检索准确性；漏检会降低召回率；误检则会增加后续存储、显示和统计模块的负担。因此，一个实用的多目标追踪系统需要在实时性、检测精度、轨迹稳定性和资源占用之间做综合取舍。 [1], [2]

0.93% (282)

2.2 Tracking-by-Detection 范式

0.09% (28)

Tracking-by-Detection 是当前多目标追踪在实现时，系统中最常用的技术范式 [1], [6]。该范式将视频目标追踪拆分为两个相对独立的阶段：首先对每一帧进行目标检测，得到目标框、类别和置信度等检测输出；随后利用追踪器对相邻帧或连续多帧中的检测输出进行关联，从而形成稳定的目标轨迹。相比端到端直接输出轨迹的方法，Tracking-by-Detection 具有工程实现清晰、模块可替换性强、便于调试和部署等优点。

0.71% (214)

本系统采用的处理路线符合典型的 Tracking-by-Detection 流程。具体到本项目，系统首先由视频读取模块从本地视频或摄像头视频源中读取帧数据，并为每一帧分配递增的 frame_id。随后，帧数据进入 YOLO 目标检测模块，由检测器输出该帧中的目标检测输出。检测输出再被送入 ByteTrack 追踪器，追踪器根据检测框位置、置信度以及运动状态预测结果进行跨帧关联，并为检测输出补充 track_id。最后，可视化与保存模块在图像上绘制检测框、类别标签、置信度、轨迹线和区域统计结果，并将处理后的帧显示或编码为输出视频。

0.88% (267)

该流程能够概括为：视频帧输入、目标检测、检测输出关联、轨迹状态更新、结果可视化输出。由于检测器和追踪器在功能上解耦，系统能够根据实际需求替换不同的目标检测模型或追踪算法。例如，在检测阶段能够使用不同规模的 YOLO 模型以适配不同硬件平台；在追踪阶段能够调整 ByteTrack 的匹配阈值和轨迹缓存长度以优化轨迹稳定性。

0.53% (161)

需要注意的是，ByteTrack 等 Tracking-by-Detection 追踪器通常默认输入帧严格遵循视频时间顺序 [5], [6]。具体到本项目，追踪器内部的卡尔曼滤波预测、轨迹生命周期更新以及检测框匹配都依赖当前帧是上一帧之后的下一时刻这一假设。如果多线程批处理系统中帧以乱序方式进入追踪器，则可能出现轨迹预测方向错误、连续帧关联失败、ID switch 增加等问题。因此，在多视频、多线程、批处理推理场景下，系统不仅要提高 GPU 推理吞吐量，还必须保证同一视频内部帧序的一致性。后文的乱序恢复机制正是针对这一工程问题设计的。

2.3 YOLO 目标检测算法

0.05% (15)

YOLO (You Only Look Once) 是一类典型的单阶段目标检测算法 [3]。其核心思想是将目标定位与类别预测整合到一次前向推理过程中，直接从输入图像预测多个候选检测框、类别概率和置信度。与两阶段检测方法需要先生成候选区域、再进行分类回归不同，YOLO 省去了复杂的候选区域生成流程，因此具备一定推理速度和较好的实时性，适合视频分析、在线监控和嵌入式部署等场景 [3], [4]。

0.64% (195)

在目标检测过程中，置信度阈值和 IoU 阈值是两个重要参数。具体到本项目，置信度阈值用于过滤模型认为可靠性较低的检测框，阈值越高，输出结果越保守，误检可能减少，但漏检风险也会增加；阈值越低，模型会保留更多候选框，有利于提高召回率，但也可能引入更多误检。IoU (Intersection over Union) 用于衡量两个检测框之间的重叠程度，常用于非极大值抑制 (Non-Maximum Suppression, NMS) 过程。当多个检测框高度重叠并指向同一目标时，NMS 会保留置信度较高的检测框并抑制冗余框，从而减少重复检测。 [3]

0.88% (267)

2.4 ByteTrack 多目标追踪算法

0.07% (21)

ByteTrack 是一种高效的多目标追踪算法，其核心思想是在目标关联时不仅使用高置信度检测框，还充分利用低置信度检测框。具体到本项目，传统 Tracking-by-Detection 方法往往只保留高置信度检测输出进行追踪，这虽然能够减少误检，但在遮挡、运动模糊、光照变化或目标尺度较小时，真实目标的检测置信度可能暂时下降。如果这些低置信度但仍然有效的检测框被直接丢弃，轨迹就容易发生断裂。ByteTrack 通过对高、低置信度检测框进行分阶段关联，在保持较高精度的同时提升了轨迹连续性。 [5]

0.82% (249)

ByteTrack 的基本流程能够分为运动预测、第一次匹配、第二次匹配和轨迹生命周期管理几个步骤 [5]。具体到本项目，首先，算法使用卡尔曼滤波对已有轨迹在当前帧中的位置进行预测。卡尔曼滤波能够根据目标历史位置和速度估计当前可能出现的位置，为后续匹配提供运动先验 [6]。随后，算法将当前帧检测框按置信度划分为高置信度检测框和低置信度检测框。第一阶段通常使用高置信度检测框与已有轨迹进行匹配，匹配依据能够包括检测框与预测框之间的 IoU 距离，并通过匈牙利算法求解全局最优匹配 [5], [6]。

0.82% (248)

经过第一阶段匹配后，部分轨迹可能仍未匹配到检测输出。ByteTrack 不立即将这些轨迹判定为丢失，而是继续使用低置信度检测框进行第二次匹配。由于低置信度框中可能包含被遮挡或检测质量较差的真实目标，该二次匹配过程能够有效减少轨迹中断。对于成功匹配的轨迹，算法会更新其位置状态和生命周期；对于未匹配轨迹，则根据丢失时间进入 `lost` 状态或最终移除；对于未匹配的高置信度检测框，算法可能初始化为新的轨迹。 [5]

0.68% (205)

2.5 优先队列与乱序恢复机制

0.05% (15)

优先队列是一种按照元素优先级而不是插入顺序取出数据的数据结构。常见实现方式是堆，其中最小堆能够保证堆顶元素始终是当前优先级最小的元素。在以`frame_id`为排序键的场景中，能够将 `frame_id` 作为优先级，把到达的帧数据压入最小堆。这样，即使帧数据进入队列的先后顺序发生混乱，系统仍然能够通过不断检查堆顶元素的 `frame_id` 来恢复时间顺序。 [8]

0.59% (179)

在多线程视频处理系统中，乱序问题较为常见。多个读入器线程可能同时从不同视频源读取帧，并竞争写入共享队列；批处理推理模块为了凑满 `batch` 可能从共享队列中按到达顺序取帧；不同视频、不同帧的读取速度、解码耗时和排队时间也可能不同 [8]。因此，共享队列中的帧顺序并不一定等同于单个视频内部的真实时间顺序。如果这些帧被直接送入 ByteTrack，追踪器就可能在第 10 帧之后接收到第 8 帧，导致运动预测与检测关联失效 [5]。

0.70% (212)

本文系统在乱序竞争批处理系统中引入了基于最小堆的帧序恢复机制。通过这种方式，系统在允许多线程乱序到达的同时，仍能保证同一视频进入追踪器前的时间顺序。

0.24% (74)

2.6 TensorRT ，浮点数格式与批处理推理

0.08% (25)

TensorRT 是 NVIDIA 面向深度学习推理优化的高性能推理框架，能够通过算子融合、层优化、精度优化和显存复用等方式提升模型在 GPU 上的执行效率 [9]。具体到本项目，在实际部署中，常见流程是先使用 PyTorch 训练或获取 `.pt` 模型，再将其导出为 TensorRT 使用的 `.engine` 文件 [10]。`.engine` 文件通常与硬件、CUDA/TensorRT 版本、输入尺寸和`batch_size`等参数相关，因此不同配置下可能需要生成不同的 `engine` 文件 [9], [10]。

0.84% (254)

FP16、`batch_size` 和输入分辨率 是影响推理速度和显存占用的关键因素。具体到本项目，FP16 半精度推理使用 16 位浮点数替代 FP32，通常能够降低显存占用并提升 Tensor Core 计算效率，但需要 GPU 硬件和模型算子支持 [9], [10]。`batch_size` 决定一次前向推理中同时处理的图像数量，较大的`batch_size`能够提高 GPU 利用率和吞吐量，但也会增加单次推理延迟和显存占用 [10]。输入分辨率 直接影响模型计算量，较高分辨率有利于检测小目标和保留细节，但会显著增加计算开销。

0.87% (263)

2.7 OpenCV 与视频处理

0.05% (16)

OpenCV 是本系统里最基础也最常用的工具之一。视频读取阶段通过 cv2.VideoCapture 接入本地文件、摄像头或网络流，并读取帧率、分辨率、总帧数等元信息；处理阶段用它完成图像缩放、文字和线条绘制、多边形 ROI 绘制、窗口显示以及输出视频编码。由于 OpenCV 返回的是 BGR 图像，系统在和 YOLO 推理接口衔接时还需要注意输入格式和尺寸转换 [11]。

0.62% (188)

在可视化部分，OpenCV 的窗口和鼠标事件机制让系统可以直接在画面上交互式圈定 ROI。用户选定区域后，程序根据目标中心点是否落入多边形来更新计数，并把检测框、追踪 ID、轨迹线和区域统计结果叠加到同一帧中。这样系统输出的不只是算法结果，也包含了面向场景分析的可读信息 [11]。

0.47% (141)

2.8 生产者-消费者模型

0.04% (13)

生产者-消费者模型是一种常见的并发程序设计模式，适用于处理速度不一致、模块职责不同的流水线系统。生产者负责生成数据并写入队列，消费者从队列中取出数据进行处理。队列在两者之间起到缓冲和解耦作用，使各个模块能够以相对独立的速度运行，降低模块之间的直接依赖。在视频处理系统中，视频读取、模型推理、目标追踪和结果保存的耗时差异明显，因此使用队列连接各阶段具有较强的工程适用性。 [8]

0.62% (188)

2.9 本章小结

0.03% (8)

本章介绍了本系统涉及的主要技术基础，包括多目标追踪任务、Tracking-by-Detection 范式、YOLO 目标检测、ByteTrack 追踪、优先队列、TensorRT 批处理推理、OpenCV 视频处理以及生产者-消费者模型。这些技术共同构成了后续系统架构设计、关键模块实现和实验分析的理论工程基础。

0.52% (157)

第 3 章 系统需求分析与总体设计

0.06% (17)

3.1 系统需求分析

0.03% (10)

本系统面向多路视频目标检测与追踪应用场景，目标是在本地视频、摄像头或网络视频流输入条件下，实现检测、追踪、轨迹可视化、区域统计、结果保存和性能监控等功能。与单视频离线处理程序相比，多路视频系统需要同时处理多个输入源，并在读取、推理、追踪、显示和保存之间保持较好的协同关系。因此，系统需求不仅包括基本功能的完整性，还包括实时性、并发处理能力、资源利用率和运行稳定性等工程指标。 [1], [2]

0.65% (196)

从功能需求来看，系统首先应支持多种视频输入形式。具体到本项目，本地视频文件适用于离线测试和实验评估，摄像头输入适用于现场实时采集，RTSP 或 HTTP 等网络视频流则适用于监控场景中的远程摄像头接入 [11], [14]。无论输入源类型如何变化，系统都需要将其统一转换为连续的视频帧，并为每一帧附加视频编号、帧编号和时间戳等元信息。其次，系统需要完成目标检测任务，利用 YOLO 模型输出目标类别、置信度和检测框坐标 [3]；随后通过 ByteTrack 对连续帧中的检测输出进行身份关联，生成稳定的 track_id [5]，并在结果画面中显示检测框、目标类别、置信度、track_id和运动轨迹。

0.99% (301)

在应用功能方面，系统还需要支持 ROI 区域计数和结果保存。具体到本项目，ROI 区域计数允许用户在实时显示窗口中通过鼠标绘制多边形区域，系统根据目标中心点或底部中心点与 ROI 的位置关系统计区域内目标数量，从而满足区域客流、车辆进入区域统计等实际需求。结果保存功能包括保存处理后的帧图像和生成带有检测框、轨迹和统计信息的输出视频，为后续分析和论文实验结果展示提供依据。此外，系统需要提供实时性能监控功能，统计每路视频的帧处理延迟、平均 FPS 等指标，辅助分析不同 batch_size、视频路数和运行策略下的性能差异。

0.86% (262)

从性能需求来看，系统需要尽可能满足实时性和低延迟要求。具体到本项目，实时性体现为系统处理速度应尽量接近或超过输入视频帧率；低延迟体现为从读取帧到输出结果之间的时间间隔应保持在可接受范围内。多路并发是本系统的重要需求，系统应能够同时处理多个视频源，并通过多线程队列和批处理推理机制提高整体吞吐量 [8]。GPU 利用率也是性能优化的关键指标，单帧逐次推理容易造成 GPU 负载不足，而多帧批处理能够提高并行计算效率 [9], [10]。与此同时，系统还应具有可扩展性，即当视频路数、输入分辨率或模型规模发生变化时，系统能够通过调整 batch_size、队列容量、推理分辨率和运行策略进行适配。

0.98% (296)

从可靠性需求来看，多线程环境下的线程安全和资源释放尤为重要。具体到本项目，系统中读入器、批处理器、Tracker、Saver 等模块通过 Queue 解耦运行，需要避免队列阻塞、线程死锁和异常退出导致的资源泄漏。实时显示窗口需要支持用户通过按键或关闭窗口主动退出，并将停止信号传递给各处理线程。对于视频读取失败、模型加载失败、推理异常、保存失败等情况，系统应通过日志记录和异常处理机制保证程序能够给出明确错误信息，并尽量完成已有资源的释放。综合来看，本系统需求能够概括为：功能完整、实时高效、并发可扩展，并在复杂运行环境下保持稳定。 [8]

0.89% (271)

3.2 系统总体架构设计

0.04% (12)

根据系统功能和处理流程，能够将本系统总体架构划分为输入层、处理层、算法层、输出层和监控层。各层之间通过统一的数据结构和队列进行连接，使视频读取、模型推理、目标追踪、结果可视化和性能统计能够相对独立地运行。分层架构的优点在于模块边界清晰，便于后续替换模型、调整处理策略和定位性能瓶颈。 [8]

0.48% (145)

输入层负责对不同类型的视频源进行统一封装。系统中 Local 视频源抽象用于处理本地视频文件，通过 OpenCV 读取视频帧、帧率、分辨率和总帧数等信息；实时视频源用于处理本地摄像头、RTSP 或 HTTP 网络视频流，并对实时视频输入进行读取和缓冲控制。通过视频源抽象抽象类，后续处理模块不需要关心具体输入形式，只需要调用统一的 open、read、close、get_fps 和 get_resolution 等接口即可获得帧数据。 [11], [14]

处理层负责组织多线程流水线和队列调度，是系统并发能力的核心。读入器从视频源读取帧并封装为帧数据对象，然后写入后续队列；批处理器或Inferencer从队列中取出帧，组织单帧或批量推理；Tracker接收带检测输出的帧并执行目标追踪；Saver负责最终结果处理和保存。队列在这些模块之间承担缓冲作用，使各阶段能够以不同速度运行，从而降低模块间的直接耦合。 [8]

0.62% (187)

算法层包含YOLO推理器和ByteTrack追踪器。具体到本项目，YOLO推理器负责目标检测推理，支持单帧单帧推理接口和批量批量推理接口两种接口 [3]，并能够根据batch_size和图片分辨率加载或导出对应的TensorRT engine [9], [10]。ByteTrack追踪器负责将YOLO检测输出转换为追踪输入，调用ByteTrack完成目标身份关联，并将track_id写回检测输出 [5]。由于每一路视频需要维护独立的追踪状态，系统在多视频模式下为每路视频创建独立的ByteTrack追踪器，避免不同视频之间的ID状态相互污染。

输出层负责将算法结果转化为可观察和可保存的形式。具体到本项目，流水线输出处理器是输出处理的统一入口，用于绘制检测框、类别、置信度、track_id和轨迹线，并根据配置保存帧图像或记录输出视频帧。实时显示模块用于多视频实时显示，支持单视频居中、双视频并排和多视频网格布局，同时支持鼠标绘制ROI区域。视频生成模块则负责将保存的帧按照指定帧率和分辨率编码为视频文件。监控层由性能监控器实现，通过探针函数在帧读取、处理完成和视频结束等关键位置记录时间，实现平均延迟、平均FPS和实时曲线显示。

0.83% (253)

整体来看，系统采用从输入层到输出层的流水线式设计：视频源首先被读取为帧数据对象，随后进入检测推理模块，再进入追踪模块，最后由输出模块完成可视化、保存和性能统计。该架构既能够支持单路视频处理，也能够扩展到多路视频并发处理；既能够运行独立流水线，也能够运行共享批处理推理模式，为后续不同策略的实验比较提供统一基础。

0.51% (156)

如图 3-1 所示，在实现时，系统总体架构借鉴分层组织与模块解耦的思想，将视频输入、处理调度、算法服务、业务功能和结果输出划分为相对独立的层次。各层之间通过帧数据对象与线程安全队列传递数据，上层业务不直接依赖底层视频源或模型实现，从而提高系统的可替换性和扩展性。 [8]

0.45% (137)

图 3-1 系统分层架构与模块化设计

0.06% (19)

3.3 多路视频并发处理中的时序一致性问题

0.07% (21)

在多路视频并发处理场景中，时序一致性是影响追踪结果正确性的关键问题。对于单个视频而言，帧的正确处理顺序应当与原始视频时间顺序一致，即frame_1、frame_2、frame_3依次进入追踪器。然而在多线程系统中，不同视频源的读取速度、解码耗时、队列等待时间和批处理调度时机并不完全一致，导致帧到达后续处理模块的顺序可能与原始时间顺序不一致。 [5], [6], [8]

0.62% (187)

多线程读取是造成乱序的直接原因之一。多个读入器线程同时运行时，每个线程读取帧的速度受视频文件编码、磁盘读取、摄像头缓冲和 CPU 调度影响。如果多个读入器竞争写入同一个共享队列，队列中的全局到达顺序只反映线程调度结果，并不能保证任意一路视频内部的帧序连续 [8]。多视频共享推理队列也会继续放大这一问题：批处理器为了尽快凑满单批次，可能将不同视频源、不同帧号的画面组合到同一次推理中。如果没有后续帧序恢复机制，同一路视频的追踪器就可能接收到非连续帧 [5]。

异步保存和显示同样会影响系统的时序表现。虽然保存模块通常位于追踪之后，但如果前面阶段已经发生乱序，Saver 只会按照接收到的顺序保存和显示结果，从而在输出视频中表现为画面跳动、轨迹不连续或 ID 显示异常。对于普通目标检测任务，乱序可能只影响输出帧排列；但对于多目标追踪任务，乱序会直接影响算法状态，因为追踪器需要依赖上一帧的轨迹状态预测当前帧目标位置。 [5], [6]

ByteTrack 等追踪器内部通常假设输入帧严格按照时间顺序到达 [5], [6]。其卡尔曼滤波预测会根据目标上一时刻位置和运动趋势估计当前时刻位置，匈牙利匹配会在预测框和当前检测框之间建立对应关系，轨迹生命周期管理也会根据连续帧中的匹配与丢失情况更新状态。如果第 10 帧先于第 8 帧进入追踪器，追踪器可能先根据较新的位置更新轨迹，再用较旧帧进行匹配，导致运动模型方向错误、轨迹断裂、ID 抖动或错误关联。

因此，时序一致性不仅是输出视频观感问题，更是系统正确性的基础约束。在多路视频系统中，应区分全局顺序和单视频内部顺序：不同视频之间的处理先后并不要求完全同步，但同一视频内部进入追踪器的帧必须尽可能保持原始时间顺序。本文后续设计的优先队列乱序恢复方案和固定取帧顺序批处理方案，都是围绕这一问题展开的。前者在乱序竞争写入场景下通过重排恢复帧序，后者则从批处理收集策略上尽量避免乱序产生。 [5], [6], [8]

3.4 基于优先队列的乱序恢复方案总体设计概述

基于优先队列的乱序恢复方案主要针对初期多线程竞争写入共享队列的处理场景。在该场景下，多个视频读取线程将 帧数据对象 写入同一个 共享队列，批处理推理模块按照到达顺序取帧并组成单批次。由于共享队列无法保证单视频内部帧序，系统需要在检测输出进入 ByteTrack 之前增加帧序恢复环节，确保每一路视频以正确顺序更新追踪状态。 [5], [8]

该机制利用最小堆的有序性恢复单视频时间顺序。由于不同视频之间各自维护独立堆结构，某一路视频的乱序等待不会直接破坏其他视频的处理逻辑。视频结束时，系统会通知对应 视频处理器 不再有新帧到达，并对堆中剩余帧按照 frame_id 顺序进行清理输出，避免尾部帧长期滞留。该方案不要求多个读入器线程同步读取，也不要求共享队列按视频顺序组织数据，因此保留了多线程竞争读取和批处理推理的灵活性。 [8]

0.62% (187)

0.68% (206)

0.67% (204)

0.08% (23)

0.56% (170)

0.64% (195)

从工程意义上看，它针对 ByteTrack 对帧序敏感的问题，在追踪器之前增加轻量级时序修正模块，使系统能够在异步读取和共享批处理条件下仍然维持单视频内部顺序。该方案的代价是需要额外维护堆缓存和期望帧号，在帧丢失或某帧长时间未到达时可能引入等待延迟。因此，在实时系统中还能够继续设计超时跳帧策略，在轨迹稳定性和实时性之间进行权衡。 [5], [8]

0.57% (174)

3.5 基于固定取帧顺序的批处理保序方案总体设计概述

0.09% (26)

为了从架构层面减少乱序恢复压力，后续系统采用了基于固定取帧顺序的批处理保序方案。与多个读入器竞争写入共享队列不同，该方案为每一路视频维护独立读取队列 [8]，批处理器 按照预先确定的视频顺序和 k 值从各视频队列中取帧，并将多路视频帧组成一个单批次 进行统一推理 [9], [10]。由于每个视频队列内部由对应读入器顺序写入，批处理器 连续取出同一路视频的若干帧时能够较好地保持该视频内部顺序。

批推理完成后，结果分发器 根据 单帧元数据 和实际取帧数量将检测输出分发回对应视频的输出队列。由于 单帧元数据 保存了每帧所属视频和帧编号，系统能够将 批量推理结果准确还原到各自视频流中。各视频后续仍然拥有独立的 ByteTrack 追踪器，因此不同视频之间的追踪状态不会相互影响 [5]。该设计既利用了 GPU 批量推理的吞吐优势，又通过固定取帧顺序和结果元信息分发降低了乱序发生概率 [9], [10]。

0.67% (204)

与优先队列乱序恢复方案相比，固定取帧顺序方案更偏向从源头控制乱序，而不是在乱序产生后再恢复。它适合作为系统的最终主处理方案，因为其结构相对清晰，推理吞吐量较高，且每路视频保持独立追踪。需要注意的是，当某一路视频读取较慢或队列暂时为空时，批次收集器 可能无法取满该路视频的目标 k 值，此时系统会根据实际取到的帧数组成单批次，从而在吞吐量和等待延迟之间保持平衡。 [8-10]

0.62% (187)

3.6 三种运行策略设计

0.04% (12)

为了比较不同并发处理方式的效果，系统提供三种运行策略作为实验对照，分别为乱序竞争批处理模式、并行流水线模式和批内分发模式。三种策略在视频读入、推理调度、追踪状态维护和帧序保障方式上各有侧重。

0.31% (95)

策略一为乱序竞争批处理模式，也称最小堆重排策略。该策略允许多个读入器竞争写入共享队列，由竞争式批处理器按batch_size取帧进行 YOLO 批量推理；推理结果再按视频源编号分发给各视频处理器，并通过最小堆按frame_id恢复单路视频内部顺序。该策略主要用于验证乱序恢复机制在异步到达场景下的作用。

0.50% (151)

策略二为并行流水线策略。该策略为每一路视频单独维护读入、推理、追踪和保存链路，各视频之间通过线程并行运行但追踪状态互不干扰。该模式结构简单、时序关系清晰，但在多路视频场景下会重复创建推理实例，可能带来额外资源占用。

0.35% (107)

策略三为批内分发策略，也是系统默认策略。该策略为每一路视频保留独立读入、追踪和保存链路，但多路视频共享一个批处理推理器。批处理器按照配额从各视频队列中取帧组成单批次，完成统一推理后再根据单帧元数据将结果分发回对应视频流。该策略兼顾批量推理效率和多路追踪状态隔离。

0.43% (131)

3.7 数据流设计

0.03% (9)

系统的数据流围绕 帧数据对象 这一统一数据结构展开。帧数据对象 是各处理模块之间传递的核心对象，用于封装一帧视频在流水线中的全部必要信息。相比直接传递图像矩阵，帧数据对象 同时携带帧序、时间戳、视频标识和检测输出，使 读入器、批处理器、Tracker、Saver 等模块能够在解耦的条件下共享上下文信息。

0.50% (152)

frame_id 是 帧数据对象 中最重要的时序字段，用于表示该帧在单个视频内部的序号。读入器 在读取每一路视频时从 1 开始递增生成 frame_id，后续乱序恢复、结果排序、帧保存命名和视频生成都依赖该字段。对于 ByteTrack 而言，frame_id 虽然不直接参与算法计算，但它决定了系统能否将帧按正确顺序送入追踪器，因此是时序一致性设计的重要基础。

0.60% (181)

时间戳 用于记录帧进入系统时的时间信息，主要服务于性能统计和延迟分析。性能监控器 在读入器读到帧后记录 start，在输出处理完成后记录 end，并在视频结束时记录 finish。通过 时间戳 和探针计时结果，系统能够统计每路视频的平均处理延迟、平均 FPS 和实时延迟变化，为后续性能实验提供数据来源。

0.50% (152)

frame 字段保存原始图像数据，通常为 OpenCV 读取到的 BGR 格式 numpy 数组。目标检测、追踪可视化、ROI 绘制和视频保存都需要使用该图像数据。视频源编号 用于区分不同视频源，通常采用 video_0、video_1 等形式；视频索引 则主要用于批处理系统中的视频索引和结果分发。二者共同保证多路视频在共享推理、统一显示和结果保存时不会发生混淆。

0.60% (183)

检测输出 字段用于保存检测与追踪结果。具体到本项目，YOLO 推理完成后，检测输出 中包含 class_id、class_name、confidence 和 检测框 等检测信息；ByteTrack 追踪完成后，系统会在原检测输出基础上补充 track_id。输出模块根据 检测输出 绘制检测框、类别标签、置信度、track_id和轨迹线，ROI 模块也会根据检测框中心点和 track_id 统计区域内目标数量。帧数据对象 将视频帧、检测输出、追踪身份和性能上下文组织为统一数据流，是系统模块化设计和多线程并发处理的基础。

3.8 本章小结

0.03% (8)

本章完成了系统需求分析与总体设计，明确了系统需要支持多路视频输入、目标检测追踪、乱序恢复、批处理推理、结果显示、区域计数和性能监控等功能。通过总体架构、数据流和三种运行策略的设计，为后续关键模块实现提供了清晰的结构依据。

0.36% (110)

第 4 章 系统关键模块设计与实现

0.06% (17)

本章在前文总体架构设计的基础上，继续说明系统各关键模块的具体实现方式。系统采用模块化设计思想，将多路视频目标追踪任务拆分为视频源抽象、帧读取与缓存、批处理推理、结果分发、目标追踪、可视化输出、ROI 区域计数和性能监控等相对独立的功能单元。各模块之间通过统一的数据结构和线程安全队列传递数据，既降低了模块之间的耦合度，也便于后续对检测模型、追踪器或显示方式进行替换和扩展。 [1], [3], [5], [8]

0.68% (205)

从工程实现角度看，本系统并不是简单地将 YOLO 检测器和 ByteTrack 追踪器串联运行，而是围绕多路视频并发处理场景中的时序一致性、批处理效率和运行可观测性进行了针对性设计。视频帧在系统中始终携带 视频源编号、frame_id、时间戳 和 检测输出 等上下文信息，使得同一帧在读取、推理、追踪、显示和保存环节中都能够被准确识别和还原。 [1], [3], [5]

0.61% (185)

4.1 视频源抽象模块设计

0.04% (13)

视频源抽象模块用于屏蔽不同输入来源之间的差异。实际应用中，系统可能面对本地视频文件、USB 摄像头、RTSP 网络流或 HTTP 视频流等多种输入形式。如果后续处理模块直接依赖具体的视频打开方式和读取方式，则系统扩展性较差，也不利于测试。为解决这一问题，本文在 视频源抽象模块 中设计了统一的视频源抽象 抽象接口。 [11], [14]

0.56% (169)

视频源抽象 对外提供 open、read、close、get_fps、get_resolution 等基本方法。读取模块只需要调用统一接口即可获得视频帧，而无需关心帧来自本地文件还是实时流。对于本地文件，视频源抽象 内部通过 OpenCV VideoCapture 打开文件并逐帧读取；对于摄像头或网络视频流，系统同样能够以统一方式封装其连接、读取和释放过程。该设计提高了视频输入层的可替换性，也为后续扩展真实摄像头部署场景提供了基础。 [11]

4.2 帧读取与队列缓存模块设计

0.05% (16)

帧读取模块是系统流水线的生产者。读入器 线程从 视频源抽象 中连续读取图像帧，并为每一帧构造 帧数据对象 对象。帧数据对象 中的 frame_id 从 1 开始递增，用于表示当前帧在单个视频内部的时间顺序；时间戳 用于记录帧进入系统的时间；视频源编号 和 视频名称 用于标识视频来源；frame 字段保存 OpenCV 读取到的图像矩阵。 [8], [11]

0.59% (179)

读入器 将构造好的 帧数据对象 写入输出队列，供后续批处理推理模块或独立流水线模块消费。当视频读取结束时，读入器 会向队列写入 None 作为结束信号，使下游模块能够判断该路视频已经处理完毕。队列缓存的引入使视频读取与后续计算过程解耦：当视频解码速度和模型推理速度存在波动时，队列能够起到缓冲作用，减少模块之间的直接阻塞。 [8]

0.54% (165)

在性能监控方面，读入器 在帧进入系统时调用 性能监控器.probe 记录 start 探针。该探针与后续输出阶段的 end 探针共同构成单帧处理延迟的统计依据。因此，帧读取模块不仅承担数据生产功能，也是系统性能观测链路的起点。

0.37% (113)

4.3 统一帧数据结构设计

0.04% (13)

帧数据对象 是系统内部最核心的数据结构。多路视频场景中，如果仅传递图像矩阵，则下游模块难以判断该帧来自哪一路视频、属于第几帧以及已经经过哪些处理环节。本文将帧图像、视频标识、frame_id、时间戳和检测追踪结果统一封装在 帧数据对象 中，使各模块在解耦运行的同时仍然共享必要上下文。 [8]

0.48% (146)

在检测完成后，YOLO 推理器会将目标类别、置信度和检测框写入 检测输出 字段；在追踪完成后，ByteTrack 追踪器 会继续为检测输出补充 track_id。具体到本项目，可视化模块根据 检测输出 绘制检测框、类别标签和轨迹线；ROI 区域计数模块根据 检测框 和 track_id 判断目标是否位于指定区域内；性能监控模块则根据 视频源编号 与 frame_id 聚合同一视频流的延迟统计。帧数据对象 将视觉内容、时序信息和处理结果组织为统一的数据载体，是系统模块化实现的基础。 [1]，[5]

0.83% (251)

如图 4-1 所示，帧数据对象 将frame_id、时间戳、视频来源、图像矩阵和检测追踪结果封装在同一对象中。该结构既为批处理分发提供 视频索引 和 视频源编号，也为乱序恢复提供 frame_id，从数据层面保证多模块协作时的上下文一致性。 [8]

0.41% (124)

图 4-1 帧数据对象 统一帧数据结构 UML 图

0.09% (26)

4.4 基于优先队列的乱序帧恢复算法设计细节

0.07% (22)

ByteTrack 追踪器默认输入视频帧按时间顺序到达。如果在多线程批处理环境中，同一路视频的第 10 帧先于第 9 帧进入追踪器，追踪器内部的运动预测和检测框关联就会失去正确的时间前提，可能导致轨迹断裂、ID 抖动或错误关联。因此，本文针对乱序竞争批处理场景设计了基于优先队列的乱序帧恢复算法。 [5]，[6]，[8]

该算法以 视频源编号 和 frame_id 为核心依据。具体到本项目，系统为每一路视频维护一个独立的最小堆，并维护 expected_frame_id 表示当前期望输出的下一帧编号。当推理后的 帧数据对象 到达时，系统首先根据 视频源编号 将其放入对应视频的最小堆。随后检查堆顶元素的 frame_id 是否等于 expected_frame_id。若相等，则说明当前期望帧已经到达，能够将该帧弹出并送入 ByteTrack；处理完成后 expected_frame_id 加一，并继续检查新的堆顶元素。若堆顶 frame_id 大于 expected_frame_id，则表示中间帧尚未到达，系统暂时保留堆中数据，等待缺失帧补齐。 [8]

由于每一路视频都拥有独立堆结构，某一路视频发生等待不会破坏其他视频的帧序管理。视频结束时，系统能够按照堆中 frame_id 顺序清理剩余帧，避免尾部帧长期滞留。该方案的时间复杂度主要来自入堆和出堆操作，单次操作复杂度为 $O(\log n)$ ，其中 n 为当前视频缓冲区中的帧数。对于实时追踪系统而言，缓冲区通常只保存短时间窗口内的少量乱序帧，因此该算法带来的额外计算开销较小。 [8]

0.63% (191)

4.5 乱序恢复算法流程

0.04% (12)

乱序恢复算法的处理流程能够概括为：首先接收乱序到达的 帧数据对象；然后依据 视频源编号 找到对应视频的最小堆；接着以 frame_id 为排序键将该帧压入堆中；之后循环检查堆顶帧号是否等于 expected_frame_id；若满足条件，则弹出该帧并输出给追踪器，同时更新期望帧号；若不满足条件，则保持等待。 [8]

0.52% (158)

在实际系统中，还需要考虑帧丢失或某一帧长时间未到达的情况。如果系统始终等待缺失帧，可能造成后续帧持续堆积并增加实时延迟。因此，乱序恢复模块能够结合超时策略使用：当等待时间超过阈值时，根据系统实时性要求选择跳过缺失帧或继续等待。对于强调轨迹稳定性的离线分析任务，能够采用更保守的等待策略；对于强调实时显示的在线监控任务，能够适当牺牲少量帧完整性以降低延迟。 [8]

0.60% (181)

算法 4-1 给出了乱序恢复模块的核心步骤。其关键在于为每一路视频维护独立最小堆和期望帧号 [8]，只在堆顶帧号与 expected_frame_id 一致时才将该帧送入 ByteTrack，从而避免同一路视频内部的时间序列被多线程调度打乱 [5]。

算法 4-1 基于优先队列的乱序帧恢复算法

0.07% (22)

4.6 多视频批处理推理模块设计

0.05% (16)

为了提升多路视频场景下的 GPU 利用率，本文设计了多视频批处理推理模块。该模块主要由配额分配、批次收集、YOLO 批量推理和结果分发等环节组成。其核心思想是将来自多路视频的帧按照一定规则组成微批次，再统一送入 YOLO 推理器执行检测。 [9], [10]

0.43% (129)

配额分配机制用于根据视频路数和batch_size计算每一路视频在一个单批次中应分配的帧数。当视频数量为 n、batch_size为 B 时，系统通常为每路视频分配 B//n 帧，并将余数分配给最后一路视频。批次收集器按照计算得到的配额依次从各视频队列中取帧，并生成单帧元数据记录视频索引、frame_id和原始帧数据对象。

0.53% (162)

批推理完成后，结果分发器 根据 单帧元数据 和实际取帧数量将检测输出写回对应 帧数据对象，并按视频索引放入各自输出队列。分发过程中，系统会对 frame metas、检测输出 和 actual_k_values 的长度进行一致性检查，若数量不一致，则按照安全长度截断并记录错误信息，避免结果错位造成视频之间检测输出混淆。该设计在提高吞吐能力的同时，保证了多路视频结果分发的准确性。

算法 4-2 展示了批处理推理后的分发策略。系统先根据视频数量和 batch_size计算每路视频的 k 值，再按固定视频顺序收集帧并记录 单帧元数据。推理完成后，结果分发器 使用 head/offset 机制将检测输出写回对应 帧数据对象，避免不同视频之间的检测输出错位。

0.45% (136)

算法 4-2 多视频批处理结果分发策略

0.07% (20)

4.7 YOLO 推理器与 TensorRT Engine 管理

0.11% (32)

YOLO 推理器是系统的目标检测封装模块，负责模型加载、TensorRT Engine 管理和批量推理结果解析。具体到本项目，系统优先根据batch_size和 图片分辨率 查找匹配的 .engine 文件。例如，默认分辨率下可使用 yolol2nb16.engine、yolol2nb32.engine 等文件；当输入分辨率为 1280 时，则使用带有 图片分辨率 后缀的 engine 文件。不同batch_size和输入分辨率对应独立 engine 文件，避免相互覆盖。 [9], [10]

当指定的 TensorRT engine 不存在时，系统会尝试从 pt 模型导出 TensorRT engine。导出过程中设置半精度、batch_size、输入分辨率和推理设备等参数，以适配当前推理配置。如果导出失败或运行环境不支持 TensorRT，系统会回退到 pt 模型执行动态batch_size推理。该策略提高了系统在不同硬件和环境下的可用性。 [9], [10]

对于 TensorRT 固定batch_size输入问题，系统采用批次填充策略。当实际收集到的帧数小于 TensorRT engine 要求的batch_size时，系统使用最后一帧进行填充，使输入数量满足 engine 约束；推理结束后，只解析真实帧对应的结果，忽略填充帧输出。这样既保证了固定batch_size engine 的可运行性，又避免填充帧对实际检测输出造成影响。 [9], [10]

4.8 ByteTrack 追踪模块设计

ByteTrack 追踪器 是系统的目标追踪封装模块。YOLO 检测输出首先被转换为 supervision.Detections 所需的数据格式，包括检测框坐标、置信度和类别编号等信息。随后，ByteTrack 追踪器 调用 ByteTrack 的 update_with_检测输出 方法完成轨迹更新，并将追踪器输出的 tracker_id 写回原检测输出。 [5]

在多路视频系统中，每一路视频都拥有独立的 ByteTrack 追踪器 实例。这样做的原因是 ByteTrack 内部维护轨迹状态、卡尔曼滤波预测状态和轨迹生命周期 [5], [6]，如果多路视频共用同一个追踪器，不同视频中的目标会被视为同一时间序列中的目标，导致 track_id 状态相互污染。独立追踪器设计使每一路视频的轨迹编号和状态更新互不影响，从而保证多视频并发处理下的追踪逻辑正确性。

图 4-2 描述了 ByteTrack 追踪模块的程序执行路径。模块首先对 YOLO 输出的字典列表进行格式转换，再调用 supervision 中的 ByteTrack 完成轨迹关联，最后将 tracker_id 写回检测结果，使后续可视化与统计模块能够基于稳定的目标编号工作。 [5]

图 4-1 ByteTrack 追踪模块程序流程图

4.9 结果可视化与轨迹绘制模块设计

可视化模块负责将检测和追踪结果以直观方式叠加到视频帧上。系统在每个检测框上绘制目标类别、置信度和 track_id，并通过不同颜色区分不同目标。为了增强轨迹分析能力，系统设计了 轨迹管理器，用于按照 track_id 维护目标历史轨迹点。 [1]

0.62% (188)

0.66% (200)

0.07% (20)

0.65% (197)

0.48% (144)

0.09% (26)

0.06% (18)

0.41% (123)

轨迹管理器 以目标底部中心点作为轨迹点位置，并记录每个轨迹点的时间戳。当同一 track_id 在连续帧中出现时，系统将新的轨迹点追加到对应 轨迹记录 中；当两次更新时间间隔超过 gap_timeout 时，系统认为轨迹出现明显中断，清空旧轨迹并重新开始记录；当某一 track_id 长时间没有更新时，系统会删除对应轨迹，防止历史轨迹长期占用内存或干扰显示。通过这种方式，系统能够在视频画面上展示目标短时间运动路径，为后续行为分析和目标检索提供可视化基础。 [1]

0.77% (233)

。

图 4-2 结果可视化与轨迹绘制程序流程图

0.07% (22)

4.10 ROI 区域计数模块设计

0.06% (17)

ROI 区域计数模块用于统计指定多边形区域内的目标数量。用户能够在实时显示窗口中通过鼠标绘制多边形区域，系统将画布坐标转换为对应视频的局部坐标，并保存为 PolygonZone。每一帧更新时，PolygonZone 根据检测框中心点判断目标是否位于多边形内部。 [13]

0.45% (135)

区域内外判断采用射线法实现。对于待判断点，从该点向一个方向作射线，统计射线与多边形边界的交点数量；若交点数量为奇数，则点位于多边形内部；若为偶数，则点位于多边形外部。系统以 track_id 为统计单位，将当前位于区域内的目标编号存入 inside_ids 集合，并以集合大小作为当前区域目标数量。与直接统计检测框数量相比，基于 track_id 的统计方式更适合连续视频场景，能够减少同一目标在连续帧中的身份混淆。 [13]

0.70% (213)

4.11 多视频实时显示布局设计

0.05% (16)

实时显示模块 模块用于多路视频的实时展示。系统根据视频路数自动选择显示布局：单视频时居中显示；双视频时采用左右并排布局；多视频时根据行列数形成网格布局。每一路视频在画布中都有对应的显示区域，系统记录该区域的 x、y、w、h 和缩放比例，用于完成视频局部坐标与显示画布坐标之间的转换。 [11]

0.48% (146)

坐标转换机制对于交互式 ROI 绘制尤为重要。用户点击显示窗口时，鼠标坐标首先属于整体画布坐标，系统需要判断该点落在哪一路视频的显示区域内，再根据缩放比例还原为视频原始坐标。只有完成这种转换，ROI 顶点才能正确对应到原始视频帧，后续区域计数结果才具有实际意义。 [11]

0.45% (136)

实时显示模块还处理用户主动关闭窗口的情況。当检测到显示窗口不可见或用户按下退出键时，系统会设置 user_quit 标志并触发 stop_event，使其他线程能够及时停止，避免窗口关闭后后台线程仍持续运行的问题。

0.35% (107)

4.12 性能监控模块设计

0.04% (13)

性能监控模块采用探针式设计，将计时逻辑插入到系统关键处理位置。读入器 读取到帧后记录 start 探针，输出处理完成后记录 end 探针，视频处理结束时记录 finish 探针。性能监控器 根据 视频源编号 和 frame_id 将同一帧的开始与结束时间进行配对，从而计算单帧端到端处理延迟。 [8]

性能显示窗口 使用独立显示线程绘制实时延迟曲线。每一路视频对应一个延迟曲线子图，图中显示当前延迟、平均延迟和历史波动情况。相比仅在程序结束后输出统计信息，实时监控能够帮助开发者观察系统运行过程中的阶段性抖动，例如 凑批等待、视频解码阻塞、显示窗口刷新或输出保存带来的延迟变化。 [8]

0.47% (142)

该模块的设计提升了系统的可观测性。后续实验中，平均延迟、实时延迟曲线和平均处理速度均可由性能监控模块提供数据支持，为不同 batch_size、不同视频路数和不同处理策略之间的对比提供依据。

0.31% (95)

4.13 配置管理与命令行接口

0.05% (15)

为了提高系统可用性和实验可复现性，系统提供了较完整的命令行配置入口。用户能够通过参数指定输入源、输出方式、模型配置、推理设备、batch_size、输入分辨率、实时显示、性能监控以及运行策略。统一入口降低了实验操作复杂度，也方便后续补充自动化测试脚本。

0.42% (126)

命令行接口使系统能够在不同实验条件下快速切换参数。例如，在测试batch_size对性能影响时，只需要修改batch_size参数即可运行不同配置；在测试输入分辨率影响时，能够切换 640 与 1280；在比较三种运行策略时，能够选择最小堆重排策略、并行流水线策略或批内分发策略。

0.46% (140)

补充配置参数，和对应功能说明

4.14 本章小结

0.03% (9)

本章围绕系统关键模块展开设计与实现说明，重点描述了视频源抽象、帧读取队列、统一帧数据结构、乱序帧恢复、批处理推理、结果分发、ByteTrack 追踪、可视化绘制、ROI 区域计数、实时显示和性能监控等模块。上述模块相互配合，使系统能够完成多路视频目标追踪的端到端运行。

0.44% (134)

第 5 章 系统测试与性能分析

0.05% (15)

本章围绕系统功能、帧序恢复、调度策略延迟、TensorRT 加速效果、输入分辨率影响以及显存占用进行实验分析。实验结果均以实际运行产生的视频输出、延迟记录、统计图和显存采样数据为依据，不再使用预期现象作为结论。

0.35% (105)

5.1 测试环境与实验数据

0.04% (13)

实验统一在同一软硬件环境下完成，主要使用 CUDA 设备进行目标检测与追踪测试。为保证实验可复现性，本文将实现依赖项和关键版本整理如表 5-1 所示。

0.25% (75)

实验内容覆盖帧序恢复效果、最小堆重排策略与批内分发策略的延迟对比、batch_size影响、TensorRT engine 与 pt 模型推理速度对比、输入分辨率对区域计数结果的影响，以及并行流水线策略与批内分发策略的显存占用对比。

0.38% (116)

表 5-1 系统实现依赖项与版本

5.2 功能测试与乱序恢复效果

0.05% (15)

功能测试首先验证系统能够完成视频读入、YOLO 检测、ByteTrack 追踪、结果绘制和视频输出等端到端流程。实验分别生成未启用帧序恢复和启用帧序恢复的对比视频，前者模拟帧或检测输出未按原始时间顺序进入追踪器的情况，后者则在追踪前依据 frame_id 对同一路视频数据进行顺序恢复 [8]，用于观察时序错乱与顺序恢复对追踪稳定性的影响 [5]，[6]。

0.59% (179)

需要说明的是，在本文当前实现的正常处理路径中，同一路视频通常由单个读取线程顺序产生 frame_id，并通过线程安全的 FIFO 队列传递，因此单一路视频自身因 race condition 直接产生帧序错乱并不是常见情况。系统更常见的并发特征是多路视频之间的到达顺序交错，以及共享批处理、异步调度或结果分发过程中需要依据 视频源编号 和 frame_id 保持每一路视频内部的时间顺序。[8]

但在真实部署环境中，单一路视频的时序抖动仍然需要被显式防范。具体到本项目，网络摄像头或 RTSP 流可能受到丢包、重传、缓存策略和低延迟解码设置的影响 [14]；硬件解码、线程池预处理、异步 GPU 推理或结果回调也可能使后帧先于前帧完成处理 [8]。如果系统按处理完成顺序而不是按显示时间戳或 frame_id 将结果送入 ByteTrack，追踪器就会在非连续时间序列上更新卡尔曼滤波状态，从而更容易出现轨迹断裂、目标编号跳变或短时间重复初始化轨迹 [5]，[6]。因此，本文引入基于优先队列的帧序恢复机制，其意义并不是假定当前串行读取链路必然发生单路乱序，而是提高系统面对真实多线程、异步推理和网络视频流场景时时序异常的鲁棒性 [8]。

1.06% (322)

图5.2.1 模拟真实环境下逻辑帧序（显示步）与真实帧序不一致

图5.2.2 乱序恢复算法确保两者一致

0.06% (19)

5.3 调度策略与单批次容量延迟分析

0.06% (18)

为便于说明实验结果，本文将平均延迟用于衡量所有记录帧的整体处理耗时，将 P95 延迟用于描述尾部延迟，即 95% 的样本延迟不超过该值。相比平均延迟，P95 更能反映实时系统中偶发卡顿和排队等待带来的最坏体验。凑批是指系统在执行一次批量推理前，需要从一个或多个视频队列中收集若干帧组成单批次；当batch_size较大或某一路视频帧到达较慢时，帧在队列中等待被收集的时间会增加。 [9]，[10]

0.66% (199)

延迟实验记录了 4 路视频源、4 种batch_size以及最小堆重排策略、批内分发策略两种调度策略下的端到端延迟。每组记录包含 120 条帧级延迟样本，表 5-2 对四路视频源的平均延迟和 P95 延迟进行汇总，数值单位均为 ms。

表 5-2 不同策略与单批次容量下的端到端延迟统计

实验结果表明，batch_size对端到端延迟的影响非常明显。具体到本项目，当batch_size为 4 时，两种策略的平均延迟均约为 80 ms，批内分发策略的平均延迟为 78.25 ms，最小堆重排策略为 79.65 ms，差异较小；当batch_size增大到 16、32 和 64 时，平均延迟快速上升，其中批内分发策略在batch_size为 64 时达到 1019.13 ms，最小堆重排策略达到 942.81 ms。说明在当前视频源数量和处理链路下，过大的batch_size会使等待凑批的时间成为端到端延迟的主要来源。

0.88% (267)

从策略差异看，最小堆重排策略在batch_size为 16、32、64 时的平均延迟分别比批内分发策略低 15.71 ms、61.56 ms 和 76.32 ms；但在batch_size为 4 时批内分发策略略低 1.40 ms。结合延迟曲线可知，若系统目标是低延迟在线处理，应优先选择较小batch_size；若后续面向离线吞吐场景，可在接受更高延迟的前提下适当增大batch_size。

从系统设计角度考虑，批内分发策略并不是“哪个视频源先有帧就先提交哪个视频源”的动态调度方式，而是先按视频源数量为每一路分配单批次内配额，再按照固定顺序逐路收集帧。实际运行时，每一路最多收集其目标配额内的帧；如果该路在限定时间内没有提供下一帧，则跳过该路剩余配额并继续尝试下一路。因此，该策略属于“定额优先、超时跳过”的收集方式，能够保证多路视频在单批次中具有相对均衡的份额，但在视频源速度不一致时，慢速视频源仍可能通过超时等待拉长本轮单批次的形成时间。

与之相比，最小堆重排策略中多个读入器竞争写入同一个共享队列，批处理器从共享队列中按到达顺序取帧组成单批次，因此更接近“先到先处理”的方式。具体到本项目，快视频源的帧不必等待固定顺序中排在前面的慢视频源，单批次更容易快速形成；随后系统再按视频源编号将推理结果分发到各自的视频处理器，并通过小顶堆按照frame_id恢复单路视频内部顺序。这解释了为什么在batch_size较大时最小堆重排策略的平均延迟和 P95 延迟通常低于批内分发策略。另一方面，当batch_size为 4 时，凑批等待本身较短，最小堆重排策略的共享队列分发和堆排序开销会抵消其调度灵活性，因此两种策略差异很小，批内分发策略甚至略低。

结合实验现象看，batch_size增大带来的延迟上升主要来自端到端流水线中的队列等待和凑批时间，而不是单次模型推理时间本身。较大的batch_size有利于提升 GPU 批处理吞吐，但在线追踪任务更关心单帧从读取到输出的响应时间，因此在当前实验条件下batch_size为 4 更适合作为实时处理配置。如果后续面向离线分析或吞吐优先场景，能够接受更高延迟并适当增大batch_size。 [9], [10]

0.67% (204)

5-1 最小堆重排策略在不同视频源和单批次容量下的延迟曲

0.10% (29)

图 5-2 批内分发策略在不同视频源和单批次容量下的延迟曲线图

0.10% (31)

5.4 TensorRT Engine 与 PT 模型推理速度对比

0.11% (33)

TensorRT 与 pt 模型对比实验通过统计图展示两种模型格式的推理速度和单帧延迟。实验结果显示，pt 模型平均推理速度为 36.68 FPS，单帧推理延迟为 27.26 ms；TensorRT engine 平均推理速度为 250.96 FPS，单帧推理延迟为 3.98 ms。 [9], [10]

0.50% (152)

该结果说明，在相同输入规模和batch_size条件下，TensorRT engine 能够显著降低模型推理开销。对于多路视频目标追踪系统而言，检测推理通常是整条流水线中最重的计算环节，因此 engine 文件带来的加速能够为多路视频并发、实时显示和后续区域统计留下更多性能余量。 [9], [10]

0.49% (150)

该差异主要来自模型执行方式的不同。具体到本项目，pt 模型运行时仍依赖通用深度学习框架完成计算图调度和算子执行，推理过程中存在较多框架层开销和动态图适配开销；TensorRT engine 则在部署前针对当前网络结构、输入尺寸和 GPU 设备完成图优化，将卷积、归一化、激活等可融合算子进行合并，并为特定硬件选择更高效的 CUDA kernel [9]。对于 YOLO 这类卷积密集型检测模型，算子融合和执行计划优化能够显著减少单帧推理耗时。因此，在相同视频输入条件下，engine 格式表现出更高 FPS 和更低单帧延迟 [10]。

0.88% (267)

需要注意的是，TensorRT 的加速收益主要体现在检测推理阶段，端到端系统延迟还会受到视频读取、批次收集、ByteTrack 更新、绘制显示和结果写入等环节影响。因此，即使 engine 将单帧推理延迟降至 3.98 ms，在batch_size较大时，系统整体延迟仍可能被凑批等待主导。这也说明模型加速和调度策略需要配合优化，单独提高推理速度并不能完全消除在线处理延迟。 [9], [10]

0.65% (197)

表 5-3 TensorRT engine 与 pt 模型推理性能对比

0.12% (35)

图 5-3 pt 模型与 TensorRT engine 的平均 FPS 对比

0.13% (39)

图 5-4 pt 模型与 TensorRT engine 的单帧延迟对比

0.12% (36)

5.5 输入分辨率对区域计数结果的影响

0.06% (19)

输入分辨率实验分别统计 640 与 1280 两种输入分辨率下的区域计数结果。实验将区域划分为“远景小目标”和“近景”两类统计区域，用于观察输入分辨率对不同尺度目标识别结果的影响。

0.30% (90)

表 5-4 不同输入分辨率下的区域计数统计

实验结果表明，提高输入分辨率对远景小目标的识别改善最明显。输入分辨率为 640 时，远景小目标平均识别数仅为 0.30，最大值为 1；输入分辨率为 1280 时，远景小目标平均识别数提高到 7.51，最大值达到 16。相比之下，近景区域平均识别数从 17.24 提高到 19.49，提升幅度相对较小。

0.49% (150)

该结果说明，高分辨率主要提升了远距离、小尺度目标的检测能力，而对近景较大目标的收益有限。同时，1280 输入分辨率实验耗时为 12.6 s，640 输入分辨率实验耗时为 11.3 s，高分辨率带来一定运行时间增加。因此，在实时监控场景中，若目标主要位于近景或对低延迟要求较高，可优先选择 640；若需要关注远景小目标或漏检代价较高，则 1280 更适合作为检测分辨率。 [3], [12]

0.64% (194)

产生该结果的原因在于输入分辨率直接决定了目标在网络输入特征图中的有效像素规模。具体到本项目，远景小目标在 640 分辨率下只占较少像素，经过网络下采样后，其边缘、纹理和形状信息更容易被压缩甚至丢失，导致置信度不足或被 NMS 过滤 [3], [12]；当输入分辨率提高到 1280 后，同一远景目标在输入图像和中间特征图上占据更多像素，检测器能够保留更多局部细节，因此远景小目标的召回明显提升。近景目标本身尺寸较大，在 640 分辨率下已经具有较充分的特征表达，因此继续提高分辨率带来的增益相对有限。

运行耗时增加则来自更大的输入张量和更多特征计算。高分辨率会提高预处理缩放、模型前向推理和后处理检测框映射的计算量，也可能增加显存读写压力。由于本文实验中近景目标检测在低分辨率下已较稳定，1280 分辨率的主要价值体现在远景小目标场景；若实际部署更强调低延迟和近景监控，640 分辨率具有更好的实时性；若任务更关注远距离目标漏检问题，则应优先采用 1280 分辨率。 [3], [12]

0.63% (192)

图 5.5.1 默认640分辨率下远景几乎无法识别目标

0.09% (27)

图 5.5.2 1280分辨率下远景识别目标能力提升

0.09% (26)

5.6 显存占用对比

0.03% (10)

显存实验按并行流水线策略到批内分发策略的顺序采样，每种策略采集 15 s，并在计算稳态平均值时排除前 3 s 的预热阶段。实验统计峰值显存增量、稳态平均显存增量和进程峰值工作集，用于比较不同推理组织方式的资源占用差异。

0.36% (109)

本节中的并行流水线策略是指每一路视频各自维护读入器、推理器、追踪器和保存器，并为每路视频创建独立的 YOLO 推理器；批内分发策略是指多路视频共享一个 YOLO 推理器和批处理推理器，再将检测输出按单帧元数据分发回对应视频流。二者的主要差异在于 GPU 侧推理实例是否按视频源重复创建。

0.47% (143)

表 5-5 并行流水线策略与批内分发策略显存占用对比

0.09% (26)

从显存增量看，并行流水线策略的峰值显存增量为 682 MiB，批内分发策略为 660 MiB，批内分发策略比并行流水线策略低 22 MiB；稳态平均显存增量方面，并行流水线策略为 671.9 MiB，批内分发策略为 649.5 MiB，批内分发策略低 22.4 MiB，并行流水线策略与批内分发策略的稳态显存比值约为 1.03。该结果支持本文的工程判断：多视频共享一个批推理实例能够减少 GPU 侧资源重复占用，而为不同视频分别创建推理实例会带来额外显存开销。 [9], [10]

产生该结果的原因在于 YOLO/TensorRT 推理实例会在 GPU 侧维护模型权重、执行上下文、输入输出缓冲区以及部分中间计算资源 [9], [10]。并行流水线策略为不同视频分别创建推理实例，虽然每一路处理逻辑更独立，但会带来一定重复资源占用；批内分发策略将多路视频集中到同一个批推理实例中，模型权重和执行上下文能够复用，因此峰值和稳态平均显存增量都略低。

本次实验中两种策略的显存差异约为 22 MiB，比例约为 3%，差距不算很大，主要与所用 yolol2n 模型规模较小、TensorRT engine 较轻量以及 CUDA /TensorRT 运行时可能共享部分底层资源有关。伴随模型规模增大、视频路数增加或每路使用更大batch_size或执行上下文时，独立实例带来的重复显存开销会更明显。 [9], [10]

0.59% (180)

5.7 实验结果综合分析

0.04% (12)

综合上述实验结果，本系统的关键结论能够概括为五点。第一，乱序恢复实验通过两段对比视频验证了帧序一致性对 ByteTrack 追踪状态更新的重要性，说明在多线程输入和批处理推理场景中必须显式维护同一路视频的时间顺序。第二，batch_size延迟实验表明，在当前实验条件下batch_size为 4 具有最低端到端延迟，继续增大batch_size会显著增加等待凑批带来的延迟，因此实时在线场景不宜盲目选择更大的batch_size。

0.71% (216)

第三，调度策略对尾部延迟具有明显影响。具体到本项目，批内分发策略采用按配额定额收集和超时跳过的方式，强调多视频源之间的配额均衡；最小堆重排策略依赖共享队列按到达顺序形成单批次，再通过小顶堆恢复单路帧序，因此在大batch_size下更容易减少慢速视频源对整体批次形成的拖累。第四，TensorRT engine 相比 pt 模型具有明显推理速度优势，FPS 从 36.68 提升到 250.96，单帧延迟从 27.26 ms 降低到 3.98 ms，是本系统获得实时处理能力的重要基础。

第五，分辨率实验说明输入分辨率为 1280 时对远景小目标检测提升明显，但会增加运行耗时；显存实验则显示批内分发策略相比并行流水线策略在峰值和稳态平均显存增量上均减少约 22 MiB。具体到本项目，结合实际实验结果，本系统在工程部署时可采用 TensorRT engine、batch_size为 4 和批内分发策略作为默认实时配置；当任务更关注低尾部延迟且能够接受视频源配额不完全均衡时，可参考最小堆重排策略的共享队列调度思路；当任务更关注远景小目标识别时，可切换到输入分辨率为 1280，但需要接受一定的额外耗时。上述实验从功能正确性、时序稳定性、推理速度、分辨率影响和资源占用多个角度验证了系统设计的有效性。

1.01% (307)

5.8 本章小结

0.03% (8)

本章依据实际实验结果，对系统功能、调度策略延迟、TensorRT 推理加速、输入分辨率影响和显存占用进行了分析。实验表明，帧序恢复能够保障追踪时序一致性，较小batch_size更适合低延迟在线处理，TensorRT engine 能明显提升推理速度，高分辨率输入有利于远景小目标识别，集中式批推理策略能够减少一定显存冗余。

0.53% (162)

第 6 章 总结与展望

0.04% (11)

6.1 工作总结

0.03% (8)

本文围绕多路视频场景下的实时目标检测与多目标追踪问题，完成了一套基于 YOLO 与 ByteTrack 的工程系统。系统以 Tracking-by-Detection 为主线[1]，在代码层面实现了视频源抽象、FrameData 帧数据结构、批次收集、结果分发、独立追踪、轨迹可视化、ROI 计数、结果保存和性能监控等模块。论文中的实验也围绕这些模块展开，而不是只停留在单模型精度比较上。

0.64% (195)

针对多线程和批处理带来的帧序问题，本文设计了基于最小堆的乱序恢复方案 [8]。该方案为每一路视频保存独立缓冲区和期望帧号，只有当堆顶帧号满足顺序要求时才送入 ByteTrack，从而减少乱序输入对轨迹连续性和 ID 稳定性的影响 [5]。针对多视频推理效率问题，系统又设计了按 k 值取帧的批内分发方案，多个视频共享一个 YOLO 推理器，再通过 FrameMeta 和 head/offset 机制把检测结果分回对应视频流。

0.70% (213)

此外，本系统实现了多视频实时显示和交互式 ROI 区域计数功能。用户能够在实时画面中观察检测框、追踪 ID 和轨迹线，并通过鼠标绘制多边形区域统计区域内目标数量。性能监控模块采用探针式设计，能够记录单帧端到端延迟并绘制实时延迟曲线，为系统调试和实验分析提供数据支持。

0.44% (133)

6.2 主要创新与亮点

0.04% (11)

本文工作的主要亮点体现在以下几个方面。第一，将帧序一致性问题纳入多目标追踪系统设计，提出并实现了基于最小堆的乱序帧恢复机制。该机制针对 ByteTrack 对时间顺序敏感的特点，在追踪器之前增加轻量级时序修正环节，使系统能够在异步读取和共享批处理条件下维持单视频内部顺序。 [5], [8]

0.48% (145)

第二，设计了面向多路视频的批处理推理与结果分发机制。系统通过配额分配机制为不同视频分配单批次内帧数，通过批次收集器按序收集多路视频帧，并通过结果分发器根据单帧元数据将检测输出写回对应视频队列。该方案既发挥了 YOLO 批量推理的吞吐优势，又避免了多视频结果混淆和追踪状态污染。 [9], [10]

0.49% (148)

第三，实现了较完整的工程化可视化与监控功能。在实现时，系统不仅输出检测框和 track_id，还维护目标历史轨迹并支持 ROI 区域计数；同时通过 性能监控器 和 性能显示窗口 对运行延迟进行实时观测。这些功能使系统从单纯算法调用扩展为可运行、可观察、可实验的多视频目标追踪平台。

0.46% (140)

6.3 不足之处

0.03% (8)

尽管本系统完成了多路视频目标追踪的主要功能，但仍存在一些不足。首先，系统主要依赖 YOLO 预训练模型进行目标检测 [3]，尚未针对特定应用场景进行数据集采集和模型微调。当场景中目标尺度较小、遮挡严重或光照变化明显时，检测输出仍可能出现漏检和误检，进而影响 ByteTrack 的追踪稳定性 [5], [12]。

0.51% (156)

其次，ByteTrack 主要依赖检测框位置、置信度和运动预测进行数据关联，缺少显式外观特征。当多个目标距离较近、长时间遮挡或交叉运动时，仍可能发生 ID switch。对于需要长期身份保持或跨摄像头目标关联的应用，仅依赖当前方案还不够充分。 [5], [7]

0.43% (130)

再次，ROI 区域目前主要通过实时交互方式绘制，缺少区域配置的持久化保存与加载机制。系统已经能够完成区域内目标数量统计，但对于越线计数、方向判断、区域停留时间统计和异常事件检测等高级语义功能支持仍然有限。

0.34% (102)

最后，本系统主要面向单机多视频处理场景，尚未扩展到分布式多摄像头管理平台。当视频源数量继续增加时，单机 GPU、视频解码、磁盘写入和显示刷新都可能成为瓶颈，需要结合任务调度、边缘计算或服务化部署继续优化。

0.34% (102)

6.4 后续展望

后续研究能够从算法增强、系统功能扩展和部署优化三个方向继续推进。在算法方面，可引入 BoT-SORT、Deep SORT 或 ReID 外观特征模块，提高目标遮挡和交叉运动场景下的身份保持能力 [7]；也能够针对具体监控场景采集数据并微调 YOLO 模型，以提升小目标、远距离目标和特定类别目标的检测效果 [3]，[12]。

在功能方面，能够在现有 ROI 区域计数基础上增加越线统计、方向判断、目标停留时间分析和异常事件检测等功能。系统还能够将检测输出、轨迹信息和区域统计结果写入数据库，支持按照时间、视频源、目标类别或 track_id 进行检索，从而继续贴近视频目标搜索和智能监控应用需求。

在系统部署方面，能够构建 Web 可视化管理界面，将视频源配置、模型参数、ROI 区域、实时结果和历史统计统一展示；也能够将推理服务、视频接入服务和结果存储服务拆分为独立模块，形成更加灵活的服务化架构。对于边缘设备部署场景，可继续采用轻量化模型、低精度推理和动态batch_size调度策略，以在有限算力条件下保持较好的实时性。 [9]，[10]

总体来看，基于 YOLO 与 ByteTrack 的多路视频目标追踪系统具有较好的工程应用价值。本文完成的系统为多视频实时分析提供了一个可运行的基础框架，后续通过补充实验数据、优化算法模块和完善应用功能，能够继续提升系统在智能安防、交通管理、校园安全和客流统计等场景中的实用性。 [1]，[2]

6.5 本章小结

本章总结了本系统的主要工作、创新点、不足与后续改进方向。总体来看，本文完成了一套可运行、可观察、可实验的多路视频目标追踪系统，为后续继续提升算法鲁棒性、扩展应用功能和优化部署方式奠定了基础。

2. 参考文献

AI特征值: 0.00% AI特征字符数/章节字符数: 0 / 0

说明:

- 1、仅支持中文内容检测;
- 2、AI特征值=AI特征字符数/总字符数;
- 3、红色代表AI特征显著部分, 计入AI特征字符数;
- 4、黄色代表AI特征疑似部分, 未计入AI特征字符数;
- 5、检测结果仅供参考, 最终判定是否存在学术不端行为时, 需结合人工复核、机构审查以及具体学术政策的综合应用进行审慎判断。

0.03% (8)

0.54% (163)

0.45% (135)

0.57% (174)

0.49% (148)

0.03% (8)

0.31% (95)